

Anbindung ServiceQ an die GITacademy

Token-Handshake, Session-Management und Referenzimplementierung

Dokumenttyp: Architektur-Challenge, Zielkonzept & technische Dokumentation

Quellsystem: ServiceQ · **Zielsystem:** GITacademy

Stand: Juli 2026 · **Status:** Entwurf zur fachlichen & Security-Abstimmung

Ehrlichkeitshinweis. Dieses Dokument kennzeichnet Annahmen mit *[Annahme]*. Als „umgesetzt“ gilt ausschließlich Code, der im Repository vorliegt; als „getestet“ gilt ausschließlich, was in der Entwicklungsumgebung tatsächlich ausgeführt wurde (siehe Abschnitt 8). DB-Atomarität, Cookie-/Redirect-Verhalten und End-to-End wurden hier **nicht** ausgeführt (keine Deno-/Postgres-Laufzeit) und gelten als offen.

1. Architektur

Das verbindliche Zielbild: Endkunden haben keinen eigenen Login in der GITacademy; der Zugriff erfolgt ausschließlich über einen Absprungpunkt in ServiceQ. ServiceQ authentifiziert den Benutzer und sich selbst per System-Credentials; die GITacademy stellt ein kurzlebiges One-Time-Launch-Ticket aus und tauscht es beim Browser-Aufruf transparent gegen eine interne Academy-Session.

[Annahme A1] „GITacademy“ = die vorliegende Codebasis: Vite + React SPA (Netlify), Backend ausschließlich Supabase (PostgreSQL, RLS, Deno Edge Functions), *kein* klassischer Applikationsserver. Diese Architektur bestimmt die Umsetzung: Der serverseitige Vertrauensanker wird als **Edge Functions + Postgres-Session/Ticket-Store** realisiert, mit einem opaquen HttpOnly- `__Host-` -Cookie – bewusst neben Supabase Auth, dessen Client-Token-Modell mehreren Anforderungen widerspricht.



2. Ablauf

1. **Absprung:** Benutzer klickt in ServiceQ „GITacademy öffnen“. ServiceQ prüft Session & Berechtigung.
2. **Ticket-Anforderung:** ServiceQ-Backend ruft `POST /academy-launch-ticket` mit Systemauthentifizierung auf.
3. **Ausstellung:** Die GITacademy validiert Client, Mandant, Rollen und Ziel, provisioniert den Benutzer (JIT) und speichert ein opaques Ticket (nur SHA-256-Hash), TTL 120 s.
4. **Weiterleitung:** ServiceQ öffnet die `launchUrl` im Browser (bevorzugt Form-POST, damit der Code nicht in der URL steht).
5. **Einlösung:** `academy-consume` entwertet das Ticket *atomar* und erstellt die Session.
6. **Session:** Set-Cookie (opaques Handle) + `303` auf das interne Ziel ohne Code.

3. Anforderungen an ServiceQ

Verantwortung	Detail
Benutzerauthentifizierung	ServiceQ authentifiziert den Endkunden; die GITacademy vertraut dieser Aussage.
Stabile Benutzer-ID	Dauerhaft unveränderliches <code>subject</code> (nicht die E-Mail). offen welche ID.
Kontextdaten	<code>tenant</code> , <code>market</code> , <code>locale</code> , <code>roles</code> , optional <code>organization</code> .
Systemauthentifizierung	[Annahme A2] <code>private_key_jwt</code> (empfohlen) – asymmetrisch signiertes Client-Assertion-JWT; Schlüssel nur im ServiceQ-Backend + Secret Store.
Ziel als ID	Nur interne Ziel-IDs (<code>target.type/id</code>), niemals freie URLs.
Idempotenz	<code>Idempotency-Key</code> pro Launch, um Ticketfluten bei Retries zu vermeiden.
Rückleitung	Fester Rückleitungspunkt je Mandant (serverseitige Config, nie aus Request).

4. API-Beispiel

4.1 Launch-Ticket erstellen

```
POST /functions/v1/academy-launch-ticket
Authorization: Bearer <system-credential>
X-Client-Id: serviceq-prod
Idempotency-Key: <uuid>
Content-Type: application/json
```

```
{
  "issuer": "serviceq",
  "subject": "8a68fd42-18fd-4a70-a933-a1a89c01ab72",
  "tenant": "PHS_AT", "market": "AT", "locale": "de-AT",
  "roles": ["service_advisor"],
  "target": { "type": "training", "id": "serviceq-basics" }
}
```

```
HTTP/1.1 201 Created
Cache-Control: no-store
```

```
{
  "launchUrl": "https://academy.example.com/functions/v1/academy-consume?code=<one-time>",
  "expiresAt": "2026-07-22T13:02:00Z",
  "correlationId": "33f8041b-3416-4c4c-a124-773388e0c207"
}
```

4.2 Einlösung (atomar)

```
UPDATE launch_ticket SET status='used', used_at=now()  
WHERE code_hash = $1 AND status='unused' AND expires_at > now()  
RETURNING *;  
-- Zwei parallele Requests: nur einer erhält die Zeile → nur eine Session.
```

5. Session-Verhalten

Einstellung	Startwert	Regel
Launch-Ticket-TTL	120 s	Ablauf gegen DB-Zeit <code>now()</code> (Clock-Skew-Schutz).
Idle-Timeout	30 min	Nur durch <i>anerkannte</i> Aktivität verlängert; reines Video/Poll zählt nicht.
Warnung	5 min vorher	Modal mit sekundengenauem Countdown, a11y, i18n.
Absolut	12 h	Ab Session-Erstellung, durch nichts verlängerbar. „24 h“ ist reine Admin-Obergrenze, keine zweite Laufzeit.
Rotation	bei Consume	Neue Session-ID; anonyme Vor-Session wird nie weiterverwendet (Fixation-Schutz).
Ablauf/Logout	serverseitig	Invalidieren + Cookie löschen + Weiterleitung auf Expired-Page mit „Zurück zu ServiceQ“ – nie zu einer Loginseite.

6. Fehlerfälle

HTTP	Code	Bedeutung
400	INVALID_REQUEST	Pflichtfeld fehlt/ungültig
401	INVALID_CLIENT	Systemauthentifizierung ungültig
403	ACCESS_DENIED	Benutzer/Rolle/Mandant/Ziel nicht erlaubt
404	TARGET_NOT_FOUND	Ziel unbekannt
409	TICKET_ALREADY_USED	Ticket bereits verwendet
410	TICKET_EXPIRED	Ticket abgelaufen
422	PROVISIONING_FAILED	Benutzer nicht anlegbar
429	RATE_LIMITED	Zu viele Anfragen
500/503	INTERNAL/UNAVAILABLE	Technischer Fehler / nicht verfügbar

Benutzer sehen nie vollständige Tokens, Stack Traces oder interne Systeminformationen – nur eine verständliche Meldung mit `correlationId` als Supportreferenz.

7. Security-Anforderungen

- **Ticket:** ≥ 256 -bit Zufall, nur SHA-256-Hash gespeichert, einmalig, kurze TTL, atomare Entwertung, keine PII im Ticket, keine vollständigen Ticketwerte in Logs.
- **Session:** opaques Handle, serverseitig; `__Host-ga_session` (HttpOnly; Secure; SameSite=Lax; Path=/); Session-ID im Browser-Storage verboten.
- **CSRF:** Double-Submit (`__Host-ga_csrf` $\leftrightarrow$ X-CSRF-Token) für alle zustandsändernden Aktionen.
- **Header:** Cache-Control: no-store , Referrer-Policy: no-referrer (Consume), X-Content-Type-Options , frame-ancestors 'none' .
- **Systemauth:** private_key_jwt /mTLS bevorzugt; Credentials pro Umgebung, rotierbar, nie im Repo/Frontend.
- **Open-Redirect-Schutz:** Ziel & Rückleitung nur aus Allowlist/Config, nie aus Request-Werten.
- **Rate-Limiting** pro Client/Subject/IP; Alerting bei Replay-/INVALID_CLIENT-Häufung.

Kritischer Bestandsbefund (P0). Die GITacademy liefert heute den Supabase-anon-Key aus, und die Demo-Policy `0002_demo_write_access.sql` erlaubt anon Voll-Lese/Schreibzugriff – *an jeder SSO-Session vorbei*. Solange der Browser einen DB-fähigen Schlüssel hält, ist der Direktzugriffs- und Bookmark-Schutz **wirkungslos**. Vor Go-Live zwingend: `0002` zurücknehmen, Fachdatenzugriff ausschließlich über session-geprüfte Endpunkte.

8. Testfälle & Testergebnisse

Tatsächlich ausgeführt in dieser Umgebung:

Testsuite	Umfang	Ergebnis
SSO-Kernlogik (<code>sso.test.ts</code> , Node 22)	23 Unit-Tests: Benutzerschlüssel, Token/Hash (SHA-256-Vektor), Request-Validierung, Rollen-Mapping (inkl. Eskalationsschutz), Zielauflösung (Open-Redirect), Locale-Fallback, Idle/Absolut-Timeout-Mathematik, Extend-Deckelung, Ticket-Fehlerklassifikation	23/23 bestanden
Frontend-Build (<code>vite build</code>)	Kompiliert inkl. SessionTimeout-Modal & SSO-Seiten	erfolgreich

Nicht ausgeführt (ehrlich gekennzeichnet): DB-Atomarität der Einlösung unter Parallelität, Cookie-/Redirect-Verhalten, End-to-End-Handshake, System-Client-Authentifizierung, Last- und Penetrationstests. Grund: In dieser Entwicklungsumgebung fehlen Deno-Runtime und Postgres. Diese Tests sind **erst in einer lauffähigen Supabase-Umgebung** möglich und gelten bis dahin als **nicht bestanden**.

Vorgesehene, noch offene Testfälle (aus dem Konzept übernommen): Ticket nach erster Verwendung/Ablauf ungültig · zwei parallele Einlösungen = eine Session · Browser-Zurück löst nicht erneut ein · ungültiges/rotiertes Client-Credential · JIT-Provisioning (neu/bestehend/doppelte E-Mail bei versch. subjects) · Autorisierung (fremder Markt/Organisation, unveröffentlicht) · Idle/Absolut/Extend/Logout · Multi-Tab · Session-Ablauf während Quiz/Video · Lasttest · Pen-Test.

9. Befundübersicht der Architektur-Challenge

ID	Prio	Thema	Kernrisiko
B1	P0	Kein Server / keine Server-Session	Konzept nicht 1:1 umsetzbar
B2	P0	anon-Key + Demo-RLS	Direktzugriffsschutz wirkungslos
B3	P1	E-Mail als Schlüssel	Account-Kollision/-Takeover
B4	P1	Schwache Systemauth (API-Key)	Identitätsübernahme bei Key-Leak
B5	P1	Idempotenz offen	Ticketflut / Races
B6	P1	Direktzugriff je Route	Bookmark-Zugriff
B7	P1	Rückleitung offen	Open Redirect
B8	P2	Video-als-Aktivität	Idle-Timeout ausgehebelt
B9	P2	12 h vs. 24 h	Timeout-Unklarheit
B10	P2	DSGVO/PII	Compliance
B11	P2	Clock-Skew	Falsch-negativ / Fenster
B12	P1	Rollen-Mapping fehlt	Privilege Escalation
B13	P3	Single-Logout	Nutzererwartung
B14	P1	OIDC-Alternative	Wartbarkeit/Standardkonformität
B15	P3	Rate-Limit/Brute-Force	DoS

10. Offene Entscheidungen & Risiken

- **P0** `0002_demo_write_access.sql` vor Go-Live zurücknehmen – sonst gesamter Handshake wirkungslos.
- **P1** **OIDC vs. bespoke Handshake** final entscheiden. Empfehlung: OIDC Authorization Code (ServiceQ/IdP als OP, GITacademy als RP), falls ein OP/IdP verfügbar ist – standardisierte Validierung, JWKS-Rotation, Back-Channel-Logout. Der bespoke Handshake ist nur gerechtfertigt, wenn ServiceQ weder OP sein kann noch hinter einem IdP steht.
- **P1** Systemauthentifizierung fixieren (`private_key_jwt` empfohlen); JWT-Verifikation im Edge-Handler ist als Erweiterungspunkt markiert und *fail-closed* – vor Produktivbetrieb ergänzen.
- **P1** Stablen `subject` aus ServiceQ verbindlich zusagen; Rückleitungsziel je Tenant konfigurieren.
- **P2** Video-als-Aktivität final auf „nein/eingeschränkt“; DSGVO: E-Mail/Name wirklich nötig? AVV Mistral; Aufbewahrungsfristen.
- **Nicht verifiziert hier:** DB-Atomarität unter Last, Cookies/Redirects, E2E, Pen-/Lasttest.

11. Umgesetzte Änderungen (Dateireferenzen)

Artefakt	Datei	Hier ausgeführt?
Architektur-Challenge (15 Befunde)	docs/serviceq-academy/01-architektur-challenge.md	n/a (Analyse)
Zielkonzept & Plan	docs/serviceq-academy/02-zielkonzept-und-plan.md	n/a (Analyse)
Schema + atomare Einlösung + RLS	supabase/migrations/0003_serviceq_sso.sql	Nein (kein Postgres)
Kernlogik (framework-neutral)	supabase/functions/_shared/sso.ts	Ja (über Node getestet)
Unit-Tests	supabase/functions/_shared/sso.test.ts	Ja – 23/23
Edge: Ticket erstellen	supabase/functions/academy-launch-ticket/index.ts	Nein (kein Deno)
Edge: Ticket einlösen (atomar)	supabase/functions/academy-consume/index.ts	Nein (kein Deno)
Edge: Session status/extend/logout	supabase/functions/academy-session/index.ts	Nein (kein Deno)
Timeout-Modal (a11y, i18n)	src/app/components/SessionTimeout.tsx	Build geprüft
SSO-Fehler-/Expired-Seiten	src/app/pages/SsoPages.tsx	Build geprüft

Erstellt als Architektur- & Security-Review der vorliegenden GITacademy-Codebasis (`sbusslehner-jpg/learning-plattform`). Vollständige Befundbegründungen (Priorität · Problem · Auswirkung · Verbesserung) in den referenzierten Markdown-Dokumenten. Dieses PDF ersetzt keine formale Datenschutz- oder Security-Freigabe.